9 ORGANIZZARE IL CODICE

OBIETTIVI

- Introdurre il concetto di separazione delle responsabilità.
- Discutere come l'ambito (scope) può controllare l'accesso alle informazioni.
- Mostrare come i moduli possono:
    - costituire l'unità organizzativa principale di un programma complesso;
    - facilitare il riutilizzo;
    - fornire interfacce e garantire l'incapsulamento delle informazioni.
- Spiegare come i pacchetti possono aggiungere ulteriori livelli di organizzazione.
- Esplorare le buone pratiche per la struttura del codice.

RIEPILOGO

Le persone che progettano le portaerei sanno come costruire sistemi modulari…
quando il bagno di una portaerei si intasa, non fa partire un missile.
Questo perché il sistema idraulico e il sistema missilistico sono ben separati.

(Tanenbaum, 2010)

Nel **Capitolo 3** abbiamo esaminato la decomposizione, un metodo che suddivide un problema in parti più piccole, ognuna delle quali può essere risolta separatamente. Abbiamo notato che esistono schemi ricorrenti nelle soluzioni, così frequenti da costituire i mattoni fondamentali di ogni linguaggio di programmazione, incluso Python.

A questo punto, dovresti avere familiarità con alcuni dei principali blocchi costitutivi del codice, tra cui:

- **Istruzioni** (espressioni e comandi);
- **Condizionali** (if, else);
- **Cicli** (for, while);
- **Sottoprogrammi** (funzioni).

Questi strumenti sono sufficienti per programmi piccoli, ma man mano che il codice cresce, è necessaria una migliore organizzazione. Un grattacielo non si costruisce con le stesse tecniche di una cuccia per cani!

INTRODUZIONE A TKINTER

Finora abbiamo visto solo programmi basati su testo, che stampano messaggi a schermo con print(). Tuttavia, la grafica introduce una nuova complessità.

Python offre il modulo **tkinter**, che permette di disegnare grafici con facilità.

Esempio di avvio con tkinter:

python

CopiaModifica

```
import tkinter


window = tkinter.Tk()
c = tkinter.Canvas(window)


c.pack()
window.mainloop()
```

Questa configurazione iniziale crea una finestra vuota. Nei prossimi esempi, aggiungeremo elementi grafici.

SEPARARE LE RESPONSABILITÀ

Il concetto di **separazione delle responsabilità** prevede che ogni parte del codice svolga un compito ben definito.

Esempio: in un foglio di calcolo, **calcolare il valore di una cella** è diverso da **mostrare il valore a schermo**.

Esempio: Disegnare una faccia

Supponiamo di voler disegnare una faccia con elementi semplici:

- Un cerchio grande per il contorno del viso.
- Due cerchi concentrici per ciascun occhio.
- Una linea per la bocca.

Ecco il codice per disegnare il viso con tkinter:

python

CopiaModifica

```
c.create_oval(50, 50, 120, 120)  # Contorno del viso
```

c.create_oval(60, 60, 80, 80) # Occhio sinistro

c.create_oval(90, 60, 110, 80) # Occhio destro

c.create_line(70, 110, 100, 110) # Bocca neutra

Ora creiamo una **funzione** per disegnare un viso neutro:

python

CopiaModifica

```
def render_neutral_face():
    c.create_oval(50, 50, 120, 120)
    c.create_oval(60, 60, 80, 80)
    c.create_oval(90, 60, 110, 80)
    c.create_line(70, 110, 100, 110)
```

Per creare una **faccia sorridente**, possiamo aggiungere un arco:

python

CopiaModifica

```
def render_smiley_face():
    c.create_oval(50, 50, 120, 120)
    c.create_oval(60, 60, 80, 80)
    c.create_oval(90, 60, 110, 80)
    c.create_arc(60, 110, 110, 80, start=0, extent=-180)  # Bocca sorridente
```

Separare i diversi aspetti aiuta a **riutilizzare e testare il codice più facilmente**.

DEFINIRE L'AMBITO DELLE INFORMAZIONI

L'**ambito** determina dove una variabile può essere usata nel codice.

- **Variabili globali**: accessibili ovunque.
- **Variabili locali**: accessibili solo all'interno di una funzione.

Esempio:

python

CopiaModifica

```
def stampa_messaggio():
```

```
    messaggio = "Ciao, mondo!"
    print(messaggio)

stampa_messaggio()
print(messaggio)  # ERRORE: "messaggio" è una variabile locale
```

L'uso eccessivo di variabili globali può generare **errori difficili da individuare**.

👉 **Soluzione**: Passare le variabili come **parametri** alle funzioni.

USARE I MODULI

Un **modulo** in Python è semplicemente un file .py che contiene codice riutilizzabile.

Esempio di modulo (modulo_utilità.py):

python

CopiaModifica

```
def somma(a, b):
    return a + b
```

Per usare questa funzione in un altro file:

python

CopiaModifica

```
import modulo_utilità
print(modulo_utilità.somma(3, 4))  # Output: 7
```

In alternativa, possiamo importare solo una funzione specifica:

python

CopiaModifica

```
from modulo_utilità import somma
print(somma(3, 4))
```

Vantaggi dell'uso dei moduli

✅ **Codice più organizzato**
✅ **Maggiore riutilizzabilità**
✅ **Facilità di manutenzione**

USARE I PACCHETTI

Se un progetto ha molti moduli, possiamo organizzarli in **pacchetti**, che sono semplicemente **cartelle contenenti più moduli**.

Struttura di esempio:

CopiaModifica

emoji/

├── drawing/

│ ├── core_draw.py

├── rendering/

│ ├── faces.py

│ ├── weather.py

│ ├── vehicles.py

Per rendere una cartella un **pacchetto Python**, deve contenere un file speciale chiamato __init__.py.

Esempio di importazione da un pacchetto:

python

CopiaModifica

import emoji.rendering.faces

emoji.rendering.faces.render_neutral_face()

CONCLUSIONE

- **Separazione delle responsabilità**: ogni funzione deve avere un compito chiaro.
- **Evitare variabili globali** per ridurre errori.
- **Usare moduli e pacchetti** per organizzare il codice in programmi complessi.